

The Kodamai Container Program: A Complete Technical Record

Part IV — Applications

MacBeth

2026-10-10

Prepared for Neil Ghani.

Abstract

Part IV records the applications of the container program: the same $\mathbf{DCont} \simeq \mathbf{Cat} + \mathbf{ZS}\text{-weld}$ machinery models real compositional systems where correctness has economic consequences. Each application is a directed container; composition is a ZS product; failure modes are obstruction classes. Every result carries its trust grade verbatim and its registry node; external/collaborator work and genuine write gaps are flagged as such.

Grade legend. `[proved]` > `[lean-verified]`/`[computed]`. Grades are copied verbatim and never inflated.

Thesis of Part IV. The same $\mathbf{DCont} \simeq \mathbf{Cat} + \mathbf{ZS}\text{-weld}$ machinery models real compositional systems where correctness has economic consequences. Each application is a directed container; composition is a ZS product; failure modes are obstruction classes.

1 Supply chains — inventory at H^1 and the gerbe at H^2

Precis. A supply chain is a small category / directed container; composing flows sharing a warehouse is a ZS product. Inventory consistency is a **degree-one** H^1 datum, INDEPENDENT of the weld class $[\omega] \in H^2$. A finer relabelling-group obstruction lives at H^2 as a gerbe.

Theorem 1.1 (Supply chain as ZS weld, `[computed]`). *A supply chain is a directed container; composing two flows = the ZS product $C \bowtie D$; the $[\omega] \in H^2$ obstruction applies. Node `supply-chain-zs`.*

Theorem 1.2 (Tier-2 inventory gerbe, `[proved]`). *When local inventory charts agree only up to an iso of stock-states (relabelling group $K = \text{Aut}(F)$), the gluing obstruction is a gerbe class $[\mathfrak{g}] \in H^2(N(C); Z(\text{Aut } F))$; it is rectifiable $\iff [\mathfrak{g}] = 0$, independent of the weld, and nonvacuous $\iff Z(\text{Aut } F) \neq 1$. Node `supply-chain-inventory-gerbe-h2`.*

Theorem 1.3 (Inventory $[\theta_R] \in H^1$, independent of the weld). *Inventory consistency is a degree-one datum $[\theta_R] \in H^1$, INDEPENDENT of the weld $[\omega] \in H^2$ (proof file `supply-chain-inventory-degree-one.tex`; appears as the N_2/H^1 rung of the nerve tower, node `composition-nerve-level`).*

Flag. This H^1 result has **no standalone registry node** of its own — it is carried inside `composition-nerve-level`. It must not be cited as an independent `[proved]` node without opening its own registry entry.

2 Blockchain / smart-contract re-entrancy

Precis. Smart contracts as coalgebras; a re-entrancy bug is a failed distributive law / an unprotected shared position in a ZS weld. The obstruction is machine-checkable.

Theorem 2.1 (Re-entrancy as failed guard, [proved]; class [lean-verified]). *Re-entrancy = a failed guard in the orchestration ZS weld (node `orchestration-zs`); the H^2 class $[\omega] = \varepsilon$ is machine-checked (`reentrancy-machine-checked-obstruction.tex` / node `fibre-h2-z2-engine`).*

Flag. Per Part II, state $[\omega]$ as a *splitting* obstruction of an associative weld, not as “non-associativity.”

Remark (Security-audit framing). State-divergence bugs are failed distributive laws (connection `security-audit-bugs-are-failed-distributive-laws`; paper `state-divergence-distributive-laws.tex`).

3 GA composition / linear attention (adjacent)

Precis. Island-model GA migration topology and linear-attention composition are instances of the same compositional algebra in $\text{Mat}(\text{Vec})$ / Poly.

Theorem 3.1 (Linear attention as $\text{Mat}(\text{Vec})$ composition, [proved]). *Linear attention = composition in the Vec-matrix bicategory $\text{Mat}(\text{Vec})$, $(P \oplus Q)(a, c) = \oplus_b P(a, b) \otimes Q(b, c)$. Node `vec-attention-composition`. (The “stack = one matrix” reading is *REFUTED — MEMORY*.)*

Remark (External / empirical). The GA diversity-dynamics empirical result (Langer–Vega, GECCO 2026) and ga-containers (ACT 2026) are external/empirical collaborator publications — cite as published collaborator work, not a MacBeth registry node.

4 Agent orchestration and tax computation

Precis. Tool-calling protocols and meta-agent orchestration are directed containers; tax computation (Catala-style) is a scope/supersession DAG with a comonadic event-sourced interface.

Remark (Tool-calling / harness). Tool-calling protocols are directed containers (shipped note `tool-calling-protocols-directed-containers.tex`; grounded in `orchestration-zs`, [proved]).

The harness-is-a-worker / MCP formalisation (paper `harness-is-a-worker.tex`; connection `harness-worker-is-t`

Flag (Tax computation — write gap). Named in SEED Path 5 but has **NO registry node and no MacBeth write-up**. Either produce the ontology-DAG / supersession-DAG / event-sourced-comonad model or mark it explicitly as future work. It is listed here as explicit future work, not as an existing result.